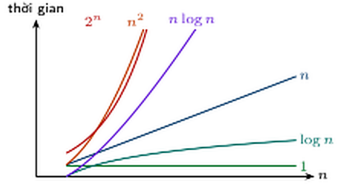
CHEATSHEET GIỮA KỲ - CẤU TRÚC DỮ LIỆU & GIẢI THUẬT

Độ phức tạp thuật toán

Chương trình = Cấu trúc dữ liệu + Giải thuật (N. Wirth)

Ý nghĩa: đo *tốc độ tăng* của thời gian / bộ nhớ khi kích thước đầu vào n tăng, không phải số giây trên một máy cụ thể.

Ký hiệu: O: cận trên (thường dùng cho xấu nhất); Ω: cận dưới; Θ: cận chặt hai phía.



$$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

Quy tắc tính

Trường hợp	Kết quả
Bỏ hằng số	$O(2n) = O(n),\ O(5n^2) = O(n^2)$
Giữ số hạng lớn nhất	$O(n^3 + 100n^2 + 5n) = O(n^3)$
Lệnh nối tiếp: *cộng*	$O(n) + O(m) = O(\max(n, m))$
Lệnh lồng nhau: *nhân*	$O(n) \cdot O(n) = O(n^2)$
Lặp phụ thuộc chỉ số	$j = i..n \Rightarrow \frac{n(n+1)}{2} = O(n^2)$
Nhân / chia hằng số	$i*=2,\ i/=3 \Rightarrow O(\log n)$
Tăng mỗi lần $\sqrt{n}$	$i = i + \sqrt{n} \Rightarrow O(\sqrt{n})$
Bước nhảy hằng số	$k+=5 \Rightarrow n/5 = O(n)$
`if/else`	nhánh tốn nhất: $O(\max(T_1, T_2))$
Gọi hàm trong vòng lặp	số vòng × chi phí hàm

Các tổng hay gặp

$1 + 2 + \cdots + n$	$\frac{n(n+1)}{2} = O(n^2)$
$1 + 2 + 4 + \cdots + n$ (cấp số nhân)	$\approx 2n = O(n)$
$\frac{n}{1} + \frac{n}{2} + \cdots + \frac{n}{n}$ (điều hòa)	$O(n \log n)$

Các dạng vòng lặp dễ nhầm

```
for(i=1;i<=n;i++) for(j=i;j<=n;j+=i) ...  // n/1+n/2+...: O(n log n)
for(i=1;i<=n;i+=2) for(j=1;j<=n;j++) ...  // log n * n = O(n log n)
for(i=1;i<=n;i++){ ...; break; }          // break ngay: O(1)
for(i=0;i<n;i++) f1(i);  f2(i);           // f2 NGOÀI vong lap
for(i=0;i<v.total();i++) ...  // total() la O(n) moi vong: O(n^2)
j=0; for(i=0;i<n;i++) while(j<n && ok) j++;  // j chi tang: O(n)
```

Hai con trỏ: `while` lồng trong `for` nhưng biến trong không bị đặt lại ⇒ tổng cộng $O(n)$ (phân tích khấu hao).

Mảng và DSLK: `v.at(i)` là $O(1)$ nhưng `list.at(i)` là $O(n)$ ⇒ duyệt DSLK bằng chỉ số tốn $O(n^2)$.

Trường hợp: tốt nhất: ít phép toán nhất; trung bình: thường gặp; xấu nhất: nhiều phép toán nhất.

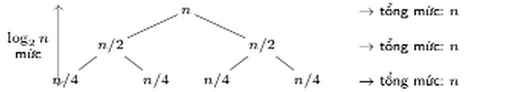
Đệ quy & hệ thức truy hồi

Cấu trúc: *trường hợp cơ bản* (điểm dừng) + lời gọi đệ quy tiến dần về điểm dừng. Không chạm được điểm dừng ⇒ đệ quy vô hạn ⇒ tràn stack.

Luôn khử được: mọi đệ quy đều viết lại được bằng vòng lặp + stack tự quản lý.

Thứ tự in: in *trước* lời gọi: xuôi ($n \to 1$); in *sau* lời gọi: ngược ($1 \to n$).

Dạng	Truy hồi	Kết quả
Chia đôi, 1 nhánh	$T(n) = T(n/2) + O(1)$	$O(\log n)$
Chia đôi, 2 nhánh	$T(n) = 2T(n/2) + O(1)$	$O(n)$
Chia để trị	$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$
Tuyến tính	$T(n) = T(n-1) + O(1)$	$O(n)$
Tuyến tính + lặp	$T(n) = T(n-1) + O(n)$	$O(n^2)$
k nhánh giảm 1	$T(n) = kT(n-1) + O(1)$	$O(k^n)$
Giai thừa	$T(n) = nT(n-1) + O(1)$	$O(n!)$



Cây đệ quy của $T(n) = 2T(n/2) + n$: $\log n$ mức × n mỗi mức $= O(n \log n)$.

Master Theorem: $T(n) = aT(n/b) + f(n)$, so $n^{\log_b a}$ với $f(n)$: lớn hơn ⇒ $O(n^{\log_b a})$; bằng nhau ⇒ $O(f(n) \log n)$; nhỏ hơn ⇒ $O(f(n))$.

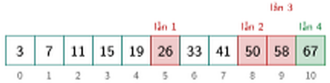
Ví dụ: $T(n) = 4T(n/2) + n$: $n^{\log_2 4} = n^2 > n \Rightarrow O(n^2)$. $T(n) = 7T(n/2) + n^2 \Rightarrow O(n^{\log_2 7})$. $T(n) = T(n/2) + n^2 \Rightarrow O(n^2)$. Số lời gọi của `f(n){ if(n<=1) return; f(n/3); }` là $\lfloor \log_3 n \rfloor + 1$.

Bẫy: hàm gọi `f(head->next)` *hai lần* trong thân ⇒ $T(n) = 2T(n-1) + c = O(2^n)$, không phải $O(n)$.

Tìm kiếm (Searching)

Kiểu	Tốt	TB	Xấu	Yêu cầu
Tuyến tính	$O(1)$	$O(n)$	$O(n)$	không
Nhị phân	$O(1)$	$O(\log n)$	$O(\log n)$	đã sắp xếp
Nhảy	$O(1)$	$O(\sqrt{n})$	$O(\sqrt{n})$	đã sắp xếp
Nội suy	$O(1)$	$O(\log \log n)$	$O(n)$	sắp xếp, phân bố đều

Tìm kiếm nhị phân



Tìm 67: so với phần tử giữa, mỗi lần loại một nửa ⇒ 4 lần so sánh.

```
int lo = 0, hi = n - 1;
while (lo <= hi) {                  // phai la <=
  int mid = lo + (hi - lo) / 2;     // tranh tran so
  if (a[mid] == x) return mid;
  if (a[mid] < x) lo = mid + 1; else hi = mid - 1;
}
return -1;
```

Số lần so sánh tối đa: $\lfloor \log_2 n \rfloor + 1$ (ví dụ $n = 1000 \Rightarrow 10$).

Bộ nhớ: bản lặp $O(1)$; bản đệ quy $O(\log n)$ do stack.

lower / upper bound: vị trí đầu tiên $\geq x$ / $> x$; hiệu của chúng là số lần x xuất hiện. Không có thì trả về n.

Bẫy: `while(lo<hi)` bỏ sót khi $lo = hi$; `lo=mid` với trung vị dưới gây lặp vô hạn; `(lo+hi)/2` có thể tràn số; trên DSLK tìm nhị phân vẫn là $O(n)$.

Tuyến tính, nhảy, nội suy

Tuyến tính: so lần lượt từ đầu, dùng được cho mảng chưa sắp xếp và DSLK. Tìm thấy trung bình $\frac{n+1}{2}$ lần so sánh, không thấy: n lần. *Lính canh:* đặt x vào cuối để bỏ kiểm tra biên.

Nhảy: nhảy từng khối $m = \sqrt{n}$ đến khi $a[\cdot] \geq x$, rồi dò tuyến tính trong khối; chi phí $\frac{n}{m} + m$. Hợp khi nhảy lùi tốn kém (băng từ).

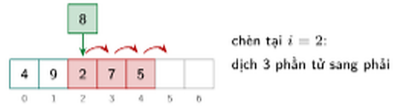
Nội suy: $pos = lo + \frac{(hi - lo)(x - a[lo])}{a[hi] - a[lo]}$. Thay tỉ số bằng $\frac{1}{2}$ thì thành nhị phân.

Bẫy: nội suy cần chặn $a[lo] \leq x \leq a[hi]$ (nếu không *pos* văng khỏi mảng) và $a[lo] \neq a[hi]$ (chia cho 0). Dữ liệu lệch như $[1, 2, \ldots, 9, 1000]$ làm nó chậm thành $O(n)$.

Mảng (Array / Array List)

Đặc điểm: các phần tử cùng kiểu nằm ở các ô nhớ **liên tiếp**; chỉ số từ 0 đến $n-1$.

Truy cập ngẫu nhiên: địa chỉ = gốc + i× kích thước phần tử ⇒ $O(1)$.



Chèn / xóa ở đầu hoặc giữa phải dịch chuyển phần tử ⇒ $O(n)$.

Thao tác	Độ phức tạp
Truy cập theo chỉ số	$O(1)$
Tìm kiếm (chưa sắp xếp / đã sắp xếp)	$O(n)$ / $O(\log n)$
Thêm / xóa ở cuối	$O(1)$
Thêm / xóa ở đầu hoặc giữa	$O(n)$

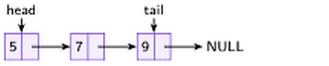
Mảng động: khi đầy thì cấp mảng mới gấp đôi rồi sao chép; thêm cuối vẫn là $O(1)$ khấu hao.

Lưu ý C++: không tự kiểm tra biên: ghi quá capacity hay lặp `i<=n` là hành vi không xác định.

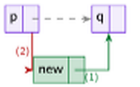
Nên dùng khi: truy cập theo chỉ số nhiều, ít chèn / xóa ở giữa; thân thiện với cache.

Danh sách liên kết đơn (Singly Linked List)

Node: dữ liệu (tùy ý) + con trỏ `next` (bắt buộc là *con trỏ*). Các node nằm rải rác trên heap, không cần khai báo trước kích thước.



Thao tác	Chỉ head	head + tail
Truy cập thứ k / tìm kiếm	$O(n)$	$O(n)$
Thêm đầu, xóa đầu	$O(1)$	$O(1)$
Thêm cuối	$O(n)$	$O(1)$
Xóa cuối (tìm node kề cuối)	$O(n)$	$O(n)$
Thêm / xóa *sau* node đã biết	$O(1)$	$O(1)$
Lấy node liền sau / liền trước	$O(1)$ / $O(n)$	



Chèn sau p: (1) new->next = p->next; rồi (2) p->next = new; – sai thứ tự là mất phần đuôi.

```
Node* reverse(Node* head){            // dao nguoc, O(n)
  Node *prev = nullptr, *cur = head;
  while(cur){ Node* nxt = cur->next;  // luu truoc khi ghi de
    cur->next = prev; prev = cur; cur = nxt; }
  return prev; }                      // prev la head moi
```

Rỗng: `head == nullptr`. Xóa node duy nhất: đặt lại *cả* `head`, `tail`, `count`.

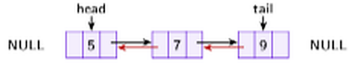
Xóa node X chỉ với con trỏ tới X: chép dữ liệu node kế tiếp vào X rồi xóa node kế tiếp, $O(1)$ (không được nếu X là node cuối).

Slow / fast: `slow` đi 1 bước, `fast` đi 2 bước: `fast` tới cuối thì `slow` ở giữa; gặp lại nhau ⇒ có vòng (Floyd).

In ngược bằng đệ quy: gọi đệ quy trước, in sau.

Bẫy: `delete head; head = head->next;` là dùng bộ nhớ đã giải phóng – phải lưu `next` trước. Xóa mọi node bằng x ở đầu danh sách phải dùng `while`, không phải `if`.

Danh sách liên kết đôi & vòng



Mỗi node có thêm prev (đỏ) ⇒ duyệt được hai chiều.

Thao tác (có head và tail)	Độ phức tạp
Truy cập phần tử thứ k / tìm kiếm	$O(n)$
Thêm, xóa ở đầu và ở cuối	$O(1)$
Xóa một node đã có con trỏ tới nó	$O(1)$
Lấy node liền trước / liền sau	$O(1)$

Chèn / xóa giữa: cập nhật **4 con trỏ**: `prev`, `next` của node mới, `next` của node trước, `prev` của node sau. Quên `prev` thì duyệt xuôi vẫn đúng nhưng duyệt ngược nhảy qua node mới.

Node giả (sentinel): `head`, `tail` là node giả nên mọi node thật đều có `prev`, `next` khác `nullptr`: chèn / xóa không cần xét trường hợp đầu, cuối, rỗng.

Danh sách vòng: node cuối trỏ về node đầu; dừng duyệt bằng `p != head` (không dùng `NULL`). DSLK đơn vòng chỉ giữ `tail`: `tail->next` là đầu ⇒ thêm cuối và xóa đầu đều $O(1)$.

Đổi lại: tốn thêm bộ nhớ cho con trỏ, phải cập nhật nhiều liên kết hơn.

CHEATSHEET GIỮA KỲ - CẤU TRÚC DỮ LIỆU & GIẢI THUẬT

Chọn Mảng hay Danh sách liên kết

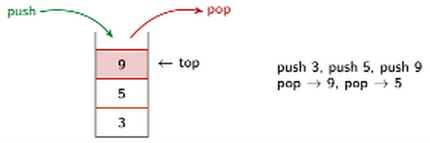
	Mảng	DSLK
Truy cập theo chỉ số	$O(1)$	$O(n)$
Chèn / xóa ở đầu	$O(n)$	$O(1)$
Chèn / xóa tại node đã biết	$O(n)$	$O(1)$
Tìm nhị phân	được	không hiệu quả
Bộ nhớ phụ	không	con trỏ mỗi node
Kích thước	cố định / nới gấp đôi	co giãn từng node

Sắp xếp DSLK: dùng **Merge Sort**: chỉ cần duyệt tuần tự, trộn bằng cách nối lại con trỏ ($O(1)$ bộ nhớ phụ), chia đôi bằng slow / fast.

Danh sách đã sắp xếp, chèn giữ thứ tự: mảng: tìm $O(\log n)$ nhưng dịch $O(n)$; DSLK: tìm $O(n)$, chèn $O(1) \Rightarrow$ cả hai đều $O(n)$.

Stack (Ngăn xếp) – LIFO

Nguyên tắc: **L**ast **I**n **F**irst **O**ut: chỉ thao tác tại một đầu (top). Là kiểu dữ liệu trừu tượng, cài bằng mảng hay DSLK đều được.



push 3, push 5, push 9
pop → 9, pop → 5

Cách cài đặt	push	pop
Mảng, đỉnh ở *cuối* mảng	$O(1)$	$O(1)$
Mảng, đỉnh ở đầu mảng (phải dịch)	$O(n)$	$O(n)$
DSLK đơn, đỉnh ở `head`	$O(1)$	$O(1)$
DSLK đơn, đỉnh ở `tail`	$O(1)$	$O(n)$
DSLK đôi, đỉnh ở đầu nào cũng được	$O(1)$	$O(1)$

top, size, empty: $O(1)$ (lưu sẵn biến đếm). Tìm kiếm: $O(n)$.

Lỗi: pop khi rỗng: *underflow*; push khi đầy: *overflow*.

Thứ tự ra: push $a_1..a_n$ rồi pop hết thì a_k ra ở lượt $n-k+1$.

Ứng dụng: DFS, Undo / Redo, nút Back, lời gọi hàm đệ quy, kiểm tra dấu ngoặc, tính biểu thức.

Ứng dụng của Stack

Dấu ngoặc: gặp ngoặc mở thì push, gặp ngoặc đóng thì pop và so khớp. Kích thước stack lớn nhất = *độ sâu lồng nhau*: `([{}])()` → 3.

Hậu tố (postfix): duyệt trái → phải; gặp số: push; gặp toán tử: pop b, pop a, push $a \circ b$.

Tiền tố (prefix): duyệt *phải* → *trái*; gặp toán tử: pop a, pop b, push $a \circ b$.

Trung tố → hậu tố: toán hạng ra thẳng; toán tử: pop các toán tử ưu tiên $\geq$ rồi push; `(`: push; `)`: pop đến `(`; cuối cùng pop hết.

Ví dụ: $(8-3)*(2+6) \to$ `8 3 - 2 6 + *` $\quad a+b*c \to$ `a b c * +`
Tính `- * 3 4 + 2 6`: push 6, 2; `+` → 8; push 4, 3; `*` → 12; `-` → $12-8=4$.

Đảo chuỗi: push hết rồi pop hết: đảo cả chuỗi; pop tại mỗi dấu cách: chỉ đảo trong từng từ. Queue rồi qua Stack = đảo ngược.

Dãy pop hợp lệ: push $1..n$ theo thứ tự, pop xen kẽ: dãy ra *không hợp lệ* khi có $i<j<k$ mà $a_k < a_i < a_j$ (mẫu "3 1 2"), ví dụ $4,3,5,1,2$.

Queue (Hàng đợi) – FIFO

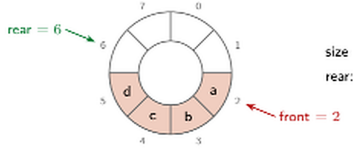
Nguyên tắc: **F**irst **I**n **F**irst **O**ut: thêm ở `rear`, lấy ở `front` (hai đầu khác nhau).

Cách cài đặt	enqueue	dequeue
Mảng thường, `front` cố định ở 0	$O(1)$	$O(n)$
Mảng vòng	$O(1)$	$O(1)$
DSLK đơn chỉ có `head`	một trong hai là $O(n)$	
DSLK đơn có `head` + `tail`	$O(1)$	$O(1)$
DSLK đơn vòng, chỉ giữ `tail`	$O(1)$	$O(1)$
DSLK đôi chỉ có `head`	$O(n)$	$O(1)$

head + tail: enqueue ở `tail`, dequeue ở `head`; làm ngược lại thì xóa `tail` tốn $O(n)$.

Ứng dụng: BFS, hàng chờ in, lập lịch CPU, bộ đệm mạng.

Hàng đợi vòng (Circular Queue)



Thao tác	Công thức
enqueue	`q[rear]=x; rear=(rear+1)%size`
dequeue	`x=q[front]; front=(front+1)%size`
Rỗng (bỏ trống 1 ô)	`front == rear`
Đầy (bỏ trống 1 ô)	`(rear+1)%size == front`
Số phần tử	`(rear-front+size)%size`

Vì sao bỏ trống 1 ô: nếu không thì đầy và rỗng đều cho `front == rear`; chứa tối đa `size-1` phần tử.

Quy ước cờ -1: khởi tạo `front = rear = -1`; rỗng khi `front == -1`; lúc này `front == rear` nghĩa là còn đúng 1 phần tử.

Ví dụ: mảng $q[13]$, `front = rear = 3`, thêm 11 phần tử: `front` giữ nguyên 3, `rear` $= (3+11) \bmod 13 = 1$.

Sắp xếp: bảng tổng hợp

Thuật toán	Tốt	TB	Xấu	Bộ nhớ	Stable
Selection	n^2	n^2	n^2	1	×
Insertion	n	n^2	n^2	1	✓
Bubble	n	n^2	n^2	1	✓
Shell	$n \log n$	$n^{3/2}$	n^2	1	×
Quick	$n \log n$	$n \log n$	n^2	$\log n$	×
Merge	$n \log n$	$n \log n$	$n \log n$	n	✓

Ổn định (stable): các phần tử bằng khóa giữ nguyên thứ tự tương đối ban đầu.

Ước lượng thời gian: $O(n^2)$: n tăng 10 lần thì thời gian tăng 100 lần.

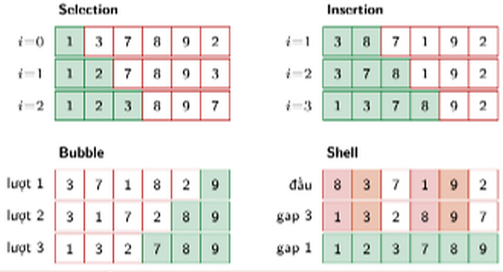
Bubble: $O(n)$ tốt nhất chỉ khi có cờ dừng sớm `swapped`.

Chọn thuật toán nào

Tình huống	Nên dùng
Mảng đã / gần sắp xếp	Insertion
Hoán đổi rất tốn kém (tối đa $n-1$ lần đổi)	Selection
Chỉ cần k phần tử nhỏ nhất: chạy k vòng, $O(kn)$	Selection
Cần ổn định và $O(n \log n)$; sắp DSLK	Merge
Dữ liệu lớn hơn RAM (sắp xếp ngoài)	Merge
Nhanh nhất trung bình trong bộ nhớ	Quick
Đã tăng dần, cần giảm dần	đảo ngược, $O(n)$

Selection, Insertion, Bubble, Shell

Cùng mảng $[8,3,7,1,9,2]$; ô xanh là phần đã vào đúng chỗ / đã có thứ tự.



Ý tưởng và dấu hiệu nhận biết	
Selection	tìm min của phần còn lại, *đổi chỗ* về vị trí i. Sau k vòng: k phần tử *đầu* đúng vị trí chung cuộc. Code có `min_idx`.
Insertion	rút `key = a[i]`, *dịch* các phần tử lớn hơn sang phải rồi chèn. Sau k vòng: $k+1$ phần tử đầu có thứ tự với nhau, phần đuôi chưa đụng tới.
Bubble	so và đổi hai phần tử *kề nhau*. Sau k lượt: k phần tử *cuối* đúng vị trí. Cờ `swapped`: một lượt không đổi chỗ thì dừng.
Shell	Insertion Sort trên các phần tử cách nhau gap; gap giảm dần $n/2, n/4, \ldots, 1$.

Số phép so sánh: Selection *luôn* $\frac{n(n-1)}{2}$; Insertion / Bubble có cờ: tốt nhất $n-1$, xấu nhất $\frac{n(n-1)}{2}$.

```
for(int i=1;i<n;i++){ int key=a[i], j=i-1;      // Insertion
  while(j>=0 && a[j]>key){ a[j+1]=a[j]; j--; }  // BBH : j>=0 truoc
  a[j+1]=key; }
```

Bẫy: Selection *không ổn định*: $[2_a, 2_b, 1] \to [1, 2_b, 2_a]$. Bubble không có cờ thì mảng đã sắp xếp vẫn tốn $O(n^2)$.

Quick Sort

Ý tưởng: chọn *pivot*, phân hoạch: nhỏ hơn sang trái, lớn hơn sang phải; đệ quy hai nửa.

Lomuto, pivot = phần tử cuối = 6
pivot về đúng vị trí chung cuộc

```
int partition(int a[], int lo, int hi){   // Lomuto
  int pivot = a[hi], i = lo - 1;
  for(int j = lo; j < hi; j++)
    if(a[j] < pivot){ i++; swap(a[i], a[j]); }
  swap(a[i+1], a[hi]);  return i + 1; }   // dung quen swap cuoi
```

Nhận ra pivot sau một lần phân hoạch: mọi phần tử bên trái nhỏ hơn và mọi phần tử bên phải lớn hơn nó; có thể có nhiều ứng viên. Một phần tử nhỏ bất thường nằm gần cuối loại hết các ứng viên bên trái nó.

Xấu nhất $O(n^2)$: pivot luôn là min / max, ví dụ lấy phần tử đầu hoặc cuối trên mảng *đã sắp xếp*; stack sâu $O(n)$ nên có thể tràn stack.

Khắc phục: pivot ngẫu nhiên, median-of-three. Nếu pivot luôn là trung vị (tìm trong $O(n)$): $T(n) = 2T(n/2) + O(n) = O(n \log n)$.

Lưu ý: phân hoạch *không* sắp xếp hai nửa; không ổn định; sắp tại chỗ nên thường nhanh hơn Merge Sort trong thực tế.

Merge Sort

Ý tưởng: chia đôi tại `mid = (left+right)/2` (không phụ thuộc giá trị), sắp từng nửa, rồi *trộn* hai nửa đã có thứ tự.

Độ phức tạp: $T(n) = 2T(n/2) + O(n) = O(n \log n)$ ở *mọi* trường hợp vì luôn chia cân bằng.

Số phần tử lẻ: 7 phần tử (0..6): `mid` = 3, nửa trái 4, nửa phải 3.

Nhược điểm: cần mảng phụ $O(n)$ để trộn (không tại chỗ).

Ưu điểm: ổn định; chỉ đọc tuần tự nên hợp DSLK và sắp xếp ngoài (dữ liệu trên đĩa, chia khối vừa RAM rồi trộn).